SIH 2026

DEVELOPER 3 — APPLICATION / BACKEND MICROSERVICE PRD

FastAPI • Auth • Business Workflows • QR • Risk • Recall

Architecture context: *This PRD defines one independently owned microservice in the agreed three-service architecture. The services are Data & Storage, Blockchain, and Application/Backend. They communicate through explicit internal contracts.*

40-hour Base Model: Build the smallest complete vertical slice first. EPCIS/event streaming, IoT, AI/ML and advanced privacy remain future modules.

1. Mission

The Application/Backend Service is the business-facing microservice. It owns FastAPI, authentication, RBAC, public API contracts, business workflows, QR, feedback, accountability, Risk Propagator and recall orchestration. The frontend talks primarily to this service.

2. Ownership

Owned by Developer 3	Explicitly not owned
FastAPI and public APIs	PostgreSQL schema
Authentication/RBAC	Redis/IPFS internals
Business workflows	Fabric network/chaincode
QR and verification workflow	Frontend UI
Feedback/accountability	Future EPCIS/streaming/IoT/AI
Risk/recall orchestration	Data-service implementation

3. Architecture

Frontend → Application Service (FastAPI) → {Data Service, Blockchain Service}

This service must never connect directly to PostgreSQL, Redis, IPFS or Fabric. It consumes their service contracts.

4. Base API domains

Domain	Capabilities
Auth	login, current user, organization/role operations
Products	create/get
Batches	create/validate/get
Units	create/get
Events	history/business events
Lineage	parents, children, full lineage
QR	resolve, generate, verify, record scan
Feedback	submit feedback, incident status
Risk	propagate risk, get scope
Recall	block, create recall action

5. Auth/RBAC

1. Authenticate protected requests.
2. Resolve actor, organization and role from trusted authentication context.
3. Enforce permissions before downstream calls.
4. Never trust client-supplied role/organization.
5. Pass actor context to Blockchain Service where required.
6. Return consumer-safe trace data.

6. First vertical slice

POST /products → RBAC/validation → Blockchain.registerProduct → COMMIT → Data Service read model
POST /batches → Blockchain.registerBatch → COMMIT → Data Service
POST /batches/{id}/validate → Blockchain.validateBatch → COMMIT → Data Service

Phase 1 exits only when Product → Batch → Validate works end-to-end through the three services.

7. Custody

Operation	Orchestration
Receive	Validate → Blockchain.receiveBatch → commit → Data Service
Transfer	Validate custodian → Blockchain.transferBatch → commit → Data Service
Transform	Validate inputs → Blockchain.transform → Data Service lineage
Create Unit	Validate batch → Blockchain.createUnit → Data Service

8. QR and scan

The Application Service owns the QR user journey. It resolves identity and history through Data Service and records the interaction through Blockchain Service. A scan is not a custody transfer.

Outer QR → resolve → retrieve history → return permitted history → Audit.recordScan()
Inner QR → verify credential → compare trusted reference/result → Audit.recordVerification()

9. Mandatory history behavior

1. Every authorized stakeholder can scan and retrieve permitted previous history.
2. The current scan is recorded at the time of scanning.
3. History includes relevant previous actors/organizations, timestamps and business-state events subject to visibility rules.
4. Scanning is separate from receiving/transferring.

10. Feedback/accountability

1. Accept feedback linked to product/unit/batch.
2. Classify issue and store description.
3. Coordinate evidence upload via Data Service.
4. Identify nearest accountable layer.
5. Aggregate similar complaints by batch/unit/category/context.
6. Apply configured escalation threshold.
7. Create/update incident and escalation state.
8. Do not treat one unverified complaint as proof of contamination.

11. Risk Propagator

Affected entity → Data Service lineage → upstream + downstream traversal → deduplicate → affected scope → persist → return

1. Trace upstream to source relationships.
2. Trace downstream to derived batches/units/locations.
3. Bound traversal and prevent loops.
4. Persist risk scope through Data Service.
5. Invalidate/recompute relevant cached results after changes.

12. Recall

1. Authorize recall/block request.
2. Define or calculate affected scope.
3. Call Blockchain Service block/recall.
4. Wait for trusted commit.
5. Persist recall read model through Data Service.
6. Return recall status and affected scope.

13. Service clients

Client	Purpose
DataServiceClient	Products, batches, units, events, lineage, incidents, evidence, risk/recall read models.
BlockchainServiceClient	Traceability, custody, transformation, audit, incident/block/recall and trusted-state queries.

14. Consistency

Client → FastAPI validation/RBAC → Blockchain Service → Fabric COMMIT → Data Service read-model update → cache invalidation → response

For ledger-governed writes, the Application Service orchestrates the cross-service workflow. It must not persist a false successful state before the trusted ledger commit.

15. Failure handling

Failure	Required behavior
Validation/RBAC failure	Reject before downstream mutation.
Blockchain rejection	Return failure; do not persist false success.
Fabric committed but Data Service failed	Keep transaction correlation and retry/reconcile; do not claim ledger rollback.
Redis unavailable	Use Data Service fallback.
IPFS upload failed	Do not finalize evidence association.

16. Mock services

Developer 3 must be able to develop before Developers 1 and 2 finish. Implement mock Data Service and Blockchain Service clients with the same contracts as the real services.

FastAPI → Mock Data Service + Mock Blockchain Service → develop/test → switch service URLs → integrate real services

17. Testing

1. API/auth/RBAC tests.
2. Business service unit tests.
3. Data/Blockchain contract tests.
4. Product → Batch → Validate integration test.
5. Receive/Transfer/Transform tests.
6. QR scan/verification tests.
7. Feedback/accountability tests.
8. Upstream/downstream Risk Propagator tests.
9. Recall orchestration tests.
10. Downstream failure/retry tests.

18. Phase 1 deliverables

1. Runnable FastAPI Application Service.
2. Authentication/RBAC skeleton.
3. Data and Blockchain service clients.
4. OpenAPI public API contract.
5. Product/Batch/Validate endpoints.
6. Standard errors and correlation IDs.
7. Health checks.
8. Mock-service development mode.
9. Tests for first vertical slice.

19. Definition of Done

The Application Service runs independently, exposes the frontend API, authenticates and authorizes users, completes Product → Batch → Validate with mocks or real services, orchestrates trusted writes through Blockchain Service, updates read models through Data Service, and never requires direct database or Fabric access.